

ME 210: Intro to Mechatronics

Lab 2

Koichi Kimoto, Julia Jiang

Part 1: Pulse Width Modulation

Q1 Source Code

```
1  #define USE_TIMER_1 true
2  #include "ISR_Timer.h"
3  #include "TimerInterrupt.h"
4  #define OUTPUT_PIN 10
5  #define POT_PIN A1
6
7  // State variables
8  volatile bool toggleState = LOW;
9  float last_freq = -1.0;
10
11 // Interrupt Service Routine to toggle the pin
12 void toggle() {
13     toggleState = !toggleState;
14     digitalWrite(OUTPUT_PIN, toggleState);
15 }
16
17 void setup() {
18     Serial.begin(9600);
19     pinMode(OUTPUT_PIN, OUTPUT);
20     ITimer1.init();
21 }
22
23 void loop() {
24     // Read Potentiometer to get duty cycle resolution
25     int val = analogRead(POT_PIN); // already 10 bit res
26
27     // hysteresis
28     if (abs(duty - val) > 2) {
29         duty = val / 1024;
30     }
31
32     ITimer1.detachInterrupt();
33     ITimer1.attachInterrupt(freq * (1 - duty), toggle);
```

C

```
34
35 // If any key pressed, stop PWM
36 if (Serial.available() > 0) {
37     ITimer1.detachInterrupt();
38 }
39 }
```

Q1.4 Waveform + Frequency

(a) What is the frequency of the duty cycle waveform that you are generating?

490Hz

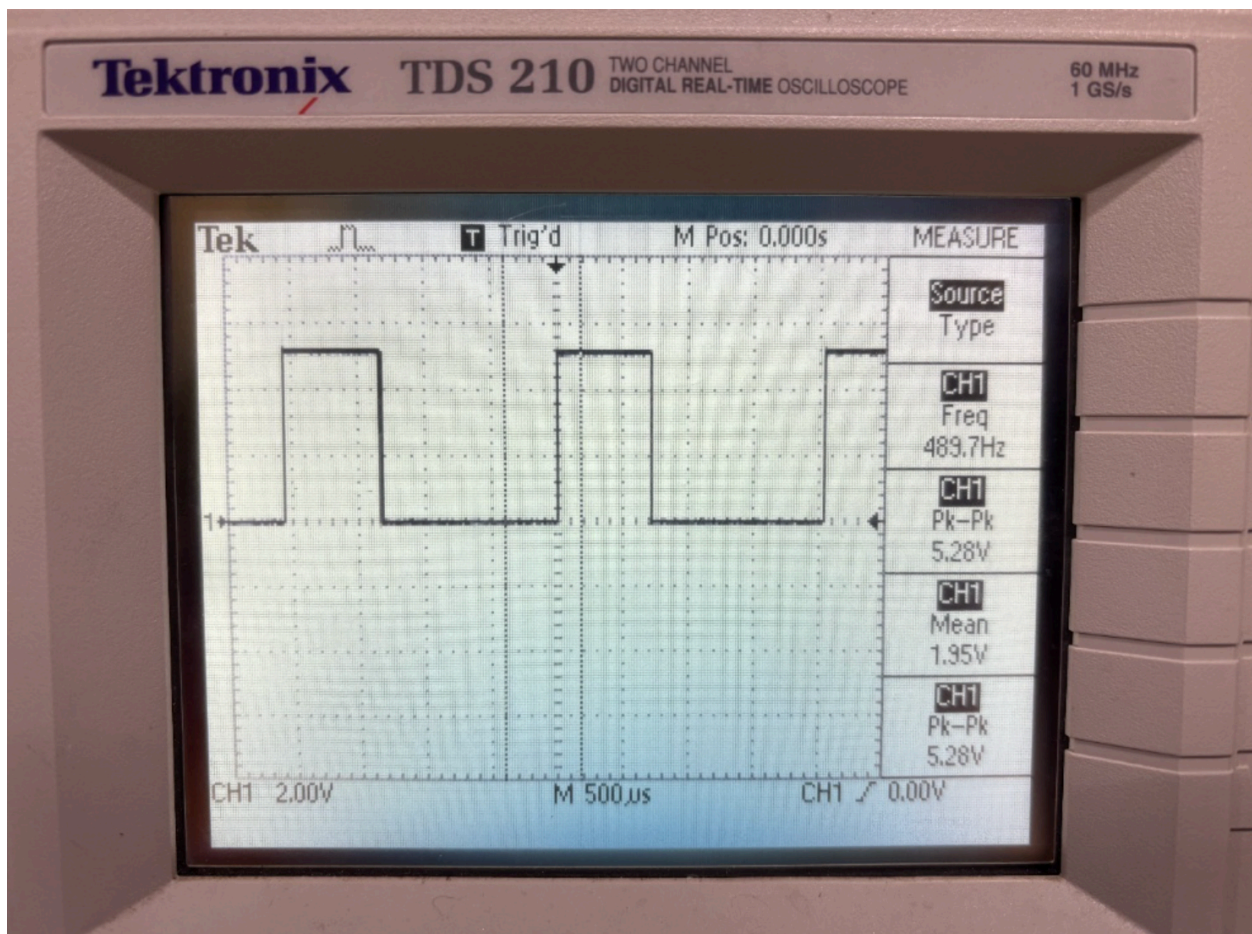


Figure 1: Oscilloscope Waveform of Duty Cycle

Q1.5 Increasing Operating Frequency

(a) Can you find a way to increase the operating frequency? What are the limitations of these methods?

We can decrease resolution in order to increase operating frequency. However, this translates to coarser control over the motor speed. Also, as frequency increases, the computer spends all its time entering and exiting the ISR and has no time to run the main loop, which causes the system to lock up.

Q1.6 Resolution vs Frequency Trade-off

- (a) What happens to the upper frequency limit if we want more duty cycle resolution? Describe the trade-off between resolution and frequency of operation.

If we want more duty cycle resolution, the upper frequency limit decreases because the hardware counter has to count higher (for more resolution) in each cycle, which takes more time and decreases the number of cycles possible in each second (frequency).

Q1.7 Full Range Operation

- (a) Can your solution achieve a full range of operation? (What happens if you try to get 0% or 100% duty cycle?) Why or why not?

At 0% or 100% duty cycle, one of the on/off intervals becomes zero, which cannot be scheduled by a timer interrupt. At the limits of 0% and 100%, the waveform has very short “blips” rather than a flat line at high or low. On the oscilloscope, as we make the duty cycle approach 0%, the output blips high and when we make the duty cycle approach 100%, the output blips at low.

Q1.8 Sources of Error

- (a) For a given duty cycle, what are the sources of error in the approach you have chosen?

It takes time to execute the code inside the toggle function, which makes the output waveform wider than calculated exactly. There

Q1.9 High PWM Frequency for Motor Control

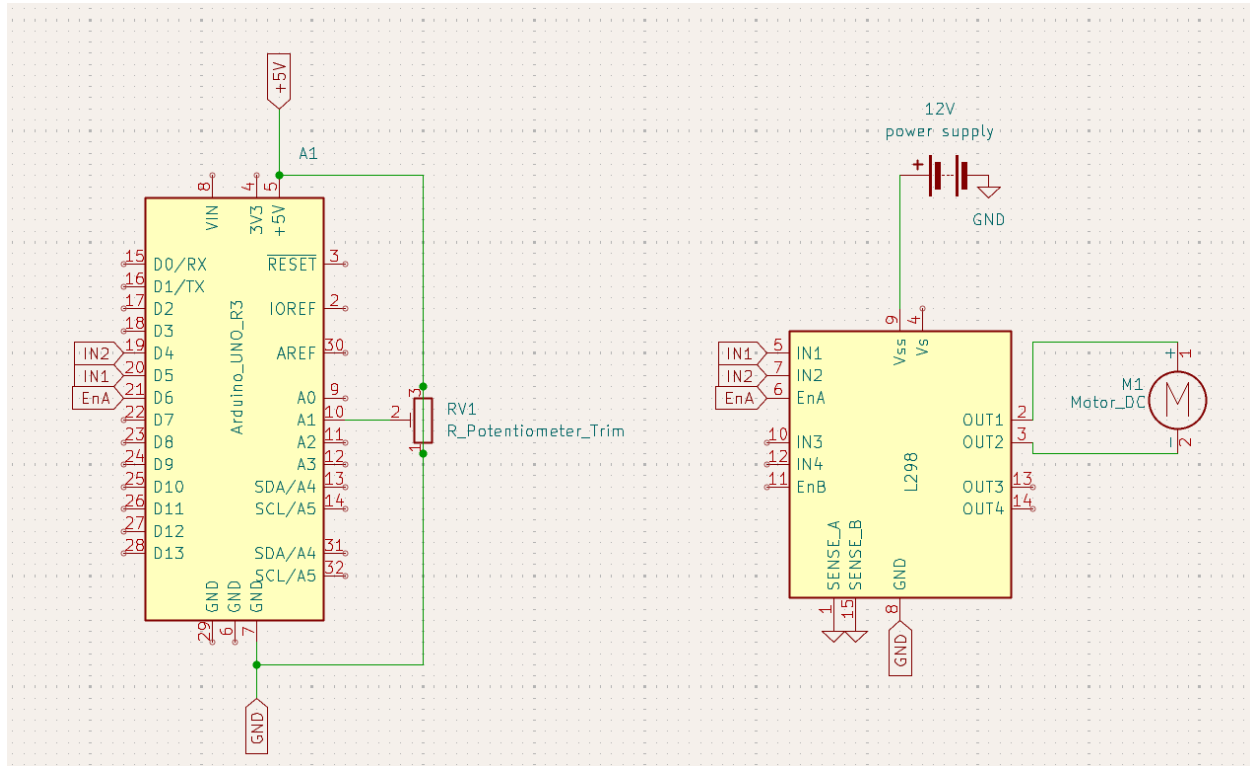
- (a) With respect to motor control, why would you want a high PWM frequency? What are the tradeoffs?

A higher PWM frequency decreases ripple because there is less time for the current to drop between pulses, but this lowers the resolution and generates more heat because the components switch on and off faster.

Part 2: Interfacing to a DC Motor and H-Bridge

Q2 Circuit Schematic

(a) Include a schematic of the circuit (indicating Arduino pins).



Q2 Source Code

(a) Include a listing of the source code used to drive the motor using the L298.

```

1  #define USE_TIMER_1 true
2  #include "ISR_Timer.h"
3  #include "TimerInterrupt.h"
4  #define OUTPUT_PIN 10
5  #define POT_PIN A1
6
7  // State variables
8  volatile bool toggleState = LOW;
9  float last_freq = -1.0;
10
11 // Interrupt Service Routine to toggle the pin
12 void toggle() {
13     toggleState = !toggleState;
14     digitalWrite(OUTPUT_PIN, toggleState);
15 }
16
17 void setup() {

```

C

```
18  Serial.begin(9600);
19  pinMode(OUTPUT_PIN, OUTPUT);
20  ITimer1.init();
21  }
22
23  void loop() {
24    // Read Potentiometer to get duty cycle resolution
25    int val = analogRead(POT_PIN); // already 10 bit res
26
27    // hysteresis
28    if (abs(duty - val) > 2) {
29      duty = val / 1024;
30    }
31
32    ITimer1.detachInterrupt();
33    ITimer1.attachInterrupt(freq * (1 - duty), toggle);
34
35    // If any key pressed, stop PWM
36    if (Serial.available() > 0) {
37      ITimer1.detachInterrupt();
38    }
39  }
```

Q2.3 Checkoff details

Checked off by: Tamir Ovedya, Noah Haile

Checked off: Jai & Taylor